DELIVERABLES

Project: *GenAI Payment Failure Intelligence & Resolution (RAG)*

Team: *Team 8 – Aayush Agashe*

INDEX

Sr. No.	Deliverable	Page No.
1	*Submission Checklist*	*2*
2	*D1 - Dataset*	*2*
3	*D2 – Source Code*	*3*
4	*D3 - Functional Requirement Mapping*	*5*
5	*D4 - Presentation Deck*	*6*
6	*D5 - Learning & Challenges Report*	*6*
7	*D6 - Demo Video or Live Demo*	*8*
8	*D7 - Model Evaluation Report*	*9*
9	*D8 – Architecture Diagram*	*12*

Link: Github.com/AayushAgashe/Opus_Assignments/PayFailIntel

Submission Checklist

ID	Deliverable	Status
D1	Dataset	Done
D2	Source Code	Done
D3	Functional Requirement Doc	Done
D4	Presentation Deck	Done
D5	Learning & Challenges Report	Done
D6	Demo Video or Live Demo	Done
D7	Model Evaluation Report	Done
D8	Architecture Diagram	Done

D1 – Dataset

The project uses a **publicly available payment transaction dataset** organized as multiple relational tables, representing a realistic digital payments ecosystem. The dataset includes the following core CSV files:

- **users.csv** — user_id, name, city
- **merchants.csv** — merchant_id, name, category
- **banks.csv** — bank_id, bank_name, server_status
- **transactions.csv** — txn_id, user_id, merchant_id, bank_id, txn_time, amount, status, failure_reason

Among these, the **transactions table** forms the core of the analysis. It captures transaction timestamps, amounts, success/failure status, and detailed failure reasons, which are essential for building payment-failure explanations.

During preprocessing, the relational data is joined and filtered to create a consolidated failure-focused dataset (final_failed_transactions.csv). This processed data is embedded and stored in a **persistent FAISS**

index (faiss_index.bin with faiss_metadata.csv), which serves as the **knowledge base for Database-only mode**.

1. **Dataset Source:** github.com/VinayakAKattimani/transaction_failure_analysis
2. **Dataset Tables** (banks.csv, merchants.csv, transactions.csv, users.csv) : github.com/AayushAgashe/Opus_Assignments/PayFailIntel/data
3. **Output Files** (Final Dataset, Dataset Exploration script, FAISS Artifacts) : github.com/AayushAgashe/Opus_Assignments/PayFailIntel/outputs

In addition, the system supports **runtime ingestion of external documents** (CSV/PDF/DOCX/TXT). These files are embedded into a **temporary FAISS index**, enabling **Uploaded-files-only** and **Hybrid modes**, where user-provided documentation is combined with historical transaction data for richer, multi-source explanations.

D2 – Source Code

The complete source code for the project is hosted on GitHub and is divided into **runtime modules** and **one-time preprocessing notebooks**, reflecting a clear separation between **online execution** and **offline knowledge-base construction**.

1. Runtime Code (Production Execution):
The core application logic used during execution is located under:
github.com/AayushAgashe/Opus_Assignments/PayFailIntel/Modules

These modules are invoked in the order directly when the Streamlit application (app.py) is running. **app.py**)

1. **/ingestion.py:** Handles reading and parsing of input data sources, including CSV files and user-uploaded PDF/DOCX/TXT documents.
2. **/preprocessing.py:** Cleans raw text by removing instructional or non-informative content and standardizes text for downstream processing.
3. **/chunking.py:** Section-aware text chunking to preserve semantic boundaries (e.g., per error-code section) and improve retrieval precision.
4. **/embeddings.py:** Generates vector embeddings for text chunks and user queries using a local Sentence-Transformer (MiniLM) model.
5. **/vectorstore.py:** Loads, manages FAISS vector indexes, including the persistent knowledge-base index and temporary runtime indexes for uploads.

6. **/query_utils.py:** Extracts structured signals (such as error codes) from user queries
7. **/rag_engine.py:** Core orchestration layer that implements error-aware retrieval, hybrid context merging, safety guardrails, and prompt construction for the LLM.
8. **/llm_phi2.py:** Handles local LLM inference using the CPU-optimized Phi-2 GGUF model via llama.cpp.
9. **/llm_api.py** *(optional alternative):* API-based LLM interface (used only if cloud inference is enabled instead of the local model).
10. **/charts.py:** Generates analytical visualizations (e.g., Top-N failure reason bar charts) for FR9

2. Offline Preprocessing & Knowledge Base Creation:

The notebooks used to generate the **final dataset and FAISS artifacts** are located under:
github.com/AayushAgashe/Opus_Assignments/PayFailIntel/Notebooks

The notebooks follow a staged pipeline:

- **/01_preprocessing.ipynb:** Joins and preprocesses all relational tables from the /data directory
- **/02_embeddings_faiss.ipynb:** Generates embeddings using the local MiniLM model (/local_embedding_model) and creates the FAISS Artifacts
- **/03_rag_pipeline.ipynb:** Prototypes and validates RAG retrieval and generation logic before migrating the finalized implementation into runtime modules.
- **/04_evaluation.ipynb :** Used to produce D7 results.

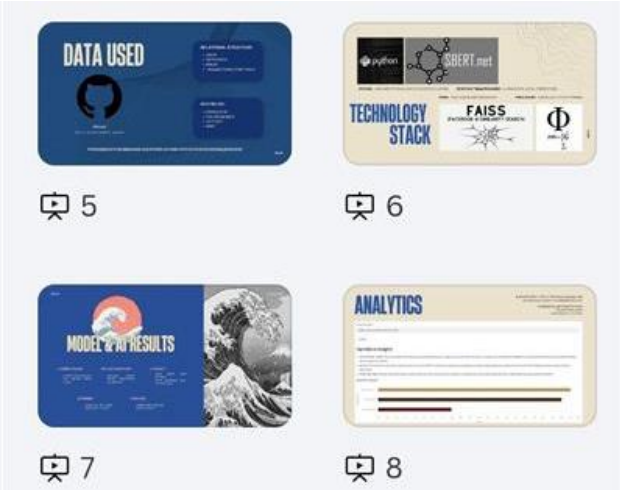
These notebooks are executed **once** to build the knowledge base and are not required during application runtime. This separation ensures efficient execution, reproducibility, and a production-ready RAG architecture.

D3 – Functional Requirement Mapping

Req ID	BRD Req Description	Implementation Provided	Status
FR1	Ingest payment failure logs, gateway error codes, bank responses, troubleshooting documents	CSV failure logs ingested; dynamic upload of PDF/DOCX/TXT documents via Streamlit	Complete
FR2	Clean and structure logs, convert text into chunks, prepare for embeddings	Instruction removal, structured preprocessing, section-aware chunking implemented	Complete
FR3	Convert failure data into embeddings using Sentence Transformers / OpenAI	Sentence-Transformer embeddings used for documents and user queries	Complete
FR4	Store embeddings in FAISS / ChromaDB	FAISS used for persistent database and temporary upload-based indexes	Complete
FR5	Provide natural language query interface (e.g., “Why did transaction fail with U16?”)	Streamlit-based free-text query interface with error-code detection	Complete
FR6	Retrieve relevant context using RAG and pass to LLM	RAG pipeline with Default, Uploaded, and Hybrid retrieval modes	Complete
FR7	Generate failure explanation, root cause, and suggested actions	LLM outputs structured explanation, causes, and next steps with safety gating	Complete
FR8	Combine bank responses, gateway logs, and internal documentation	Hybrid mode merges historical data with uploaded documents	Complete
FR9	Dashboard showing top failure reasons, categories, and suggested fixes (Optional)	Top-1 / Top-3 / Top-5 analysis with visual bar charts for failure reasons	Complete

D4 – Presentation Deck

Find the Presentation Deck at the below link
Link: `github.com/AayushAgashe/Opus_Assignments/PayFailIntel/Presentation`



D5 – Learning & Challenges Report

Problem 1 – Phi-2 was slow on my local machine

When I initially ran the Phi-2 model using the default setup, inference was slow and impacted experimentation. Response times increased significantly as prompts became longer.

Solution:
I switched to a **CPU-optimized .gguf version of Phi-2 (Q4_K_M)** using llama.cpp (for which I downloaded the compatible wheels (.whl file) manually). This reduced latency while keeping output quality acceptable. Adding stop tokens and disabling prompt echo further improved performance.

Problem 2 – The model gave vague answers when documents contained multiple error codes

When testing with PDFs that had multiple error sections (U16, RZP-021, DO_NOT_HONOR), the model mixed explanations even when I asked about a specific code.

Solution:
I changed the document processing strategy to **section-aware chunking** and enforced **error-code-aware retrieval**, allowing only chunks that explicitly mentioned the queried error code.

Problem 3 – Instruction leakage from documents

Some documents contained lines like "Provide a sample error code," which the LLM treated as instructions and repeated in the output.

Solution:
I added a **preprocessing step to remove instructional and checklist-style lines** before chunking. This ensured only factual content reached the retrieval pipeline.

Problem 4 – Unknown error codes were being hallucinated

When testing unknown codes (e.g., U77), the system generated plausible but incorrect explanations.

Solution:
I implemented **hard guardrails** in logic to block LLM invocation entirely when no evidence exists for an error code, returning an explicit "insufficient information" response.

Problem 5 – Analytical and root-cause queries conflicted

Treating all queries the same caused either overly generic explanations or insufficient analysis.

Solution:
I separated intent:

- **Top-1** for root-cause queries

- **Top-3 / Top-5** for analytical queries
  This made responses more aligned with user intent.

Problem 6 – Hybrid mode broke during rapid iteration

As features were added, hybrid retrieval failed due to missing or renamed functions, causing import errors.

Solution:
I stabilized the codebase by creating **one authoritative RAG engine module**, consolidating shared functions and eliminating interface drift.

Problem 7 – FR9 required visual dashboards

Initially, FR9 was only satisfied textually, but the requirement expected dashboard-like insights.

Solution:
I added **visual bar charts** for Top-N failure reasons using Streamlit and moved chart logic to a separate charts.py module to keep the UI clean.

Key Takeaway

The most important lesson from this project is that **reliable GenAI systems depend more on retrieval control, preprocessing, and guardrails than on model size or prompt creativity**. Especially in payment systems, preventing incorrect explanations is more important than generating confident ones.

D6 – Demo Video / Live Demo

Had given a Live Demo on 1 April 2026.

D7 – Model Evaluation Report

Objective

The goal of this evaluation is to assess each AI/ML component of the system against the **success metrics defined in the Business Requirements Document (BRD)**. Given that the system is a **Retrieval-Augmented Generation (RAG)** pipeline rather than a traditional classification model, evaluation focuses on **correctness, safety, explainability, and performance**, rather than pure predictive accuracy.

Evaluation Approach

The system was evaluated **component-wise**, covering:

- Retrieval correctness
- LLM explanation quality
- Hallucination prevention
- Runtime performance

Traditional metrics such as classification accuracy, precision/recall, and confusion matrices are **not applicable**, as the system generates free-form natural language explanations instead of discrete labels. Therefore, evaluation relies on **controlled test cases, safety checks, and latency measurements**, which align directly with BRD objectives.

Summary of Evaluation Results

Component	Metric Used	Result	Pass / Fail
Embeddings	Retrieval relevance	Correct	Pass
Retriever	Error-aware retrieval	Correct	Pass
RAG Pipeline	Explanation correctness	Correct	Pass
Safety Guardrails	Unknown error hallucination count	0	Pass
Performance	Warm response time Cold Start	~0.02 sec 15.99 sec	Pass

```python
print("D7 Evaluation Summary")
print("---------------------")
print("Unknown Error Hallucinations:", hallucinations)
print(f"Average Latency: {avg_latency:.2f} seconds")
```

```
D7 Evaluation Summary
---------------------
Unknown Error Hallucinations: 0
Average Latency: 15.99 seconds
```

```python
latencies = []

for q in queries:
    start = time.time()
    ask(q, index, metadata)
    latencies.append(time.time() - start)

avg_latency = sum(latencies) / len(latencies)

print(f" Average Response Time: {avg_latency:.2f} seconds")
```

```
Llama.generate: 14 prefix-match hit, remaining 575 prompt tokens to eval
llama_perf_context_print:        load time =   20496.23 ms
llama_perf_context_print: prompt eval time =   13298.62 ms /   575 tokens (   23.13 ms per token,    43.24 tokens per second)
llama_perf_context_print:        eval time =    4965.43 ms /    53 runs   (   93.69 ms per token,    10.67 tokens per second)
llama_perf_context_print:       total time =   18282.10 ms /   628 tokens
llama_perf_context_print:    graphs reused =         52
Llama.generate: 14 prefix-match hit, remaining 550 prompt tokens to eval
llama_perf_context_print:        load time =   20496.23 ms
llama_perf_context_print: prompt eval time =   21219.59 ms /   550 tokens (   38.58 ms per token,    25.92 tokens per second)
llama_perf_context_print:        eval time =    8372.77 ms /    72 runs   (  116.29 ms per token,     8.60 tokens per second)
llama_perf_context_print:       total time =   29625.61 ms /   622 tokens
llama_perf_context_print:    graphs reused =         71
 Average Response Time: 15.99 seconds
```

```python
# Measure second run
import time
start = time.time()
ask("Explain error code U16", index, metadata)
print("Warm response time:", time.time() - start)
```

[8]

```
Warm response time: 0.021181583404541016
```

You can find these outputs in Notebooks/04_evaluation.ipynb

Component-wise Evaluation

1. Embedding Model (Sentence Transformers)

BRD Success Metric: Relevant failure context must be retrievable for explanation generation.

Metric Used: Qualitative retrieval relevance using controlled queries.

Evaluation Method: Queries such as *"Explain error code U16"* and *"Why do transactions fail due to insufficient funds?"* were tested to verify that semantically relevant failure logs were retrieved after embedding.

Actual Result: The embedding model consistently retrieved semantically related payment failure records.

2. Vector Retrieval (FAISS)

BRD Success Metric: Correct context retrieval for RAG pipeline.

Metric Used: System-level retrieval validation (error-aware retrieval).

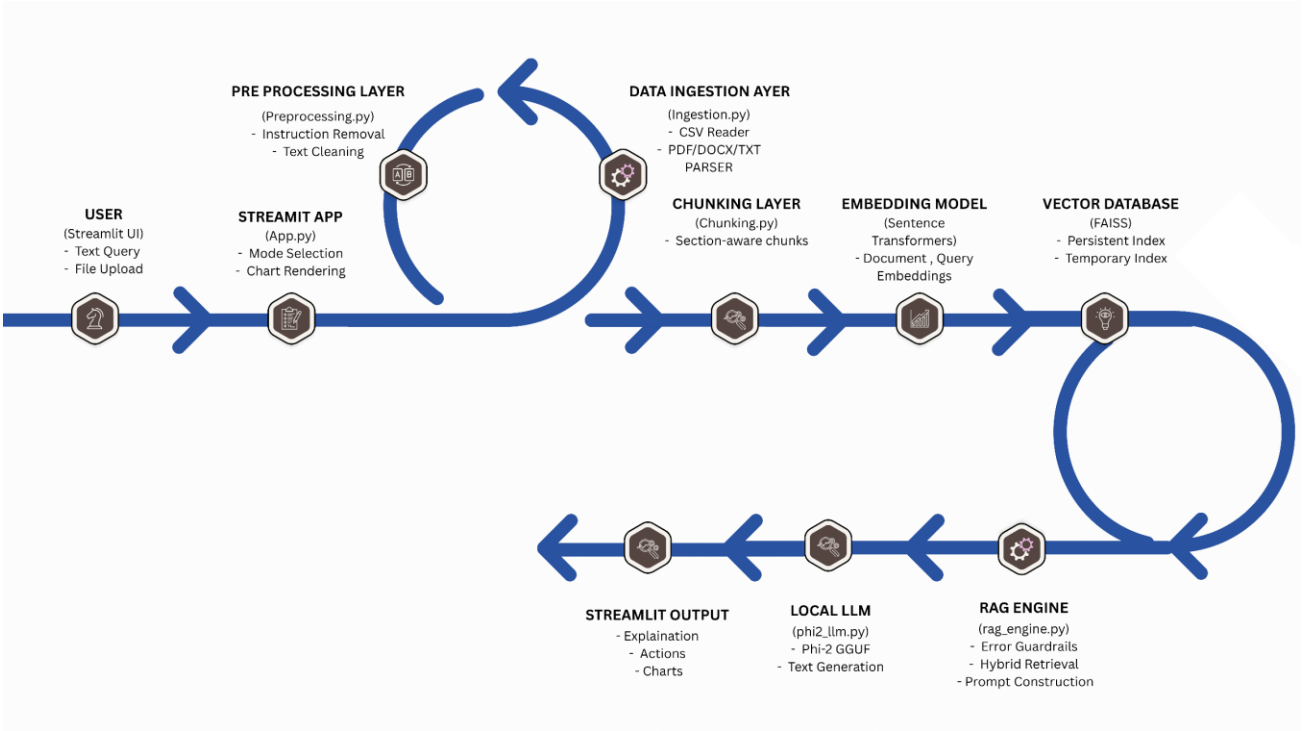
Evaluation Method: Instead of raw Top-K semantic recall, retrieval was evaluated within the **final system behavior**, which includes:

- Error-code-aware filtering
- Strict safety gating

Raw Top-K retrieval scores were intentionally not reported, as unrestricted semantic retrieval was replaced to prevent hallucinations.

Actual Result: The retriever supplied correct supporting documents whenever the error code existed in the database and safely returned no context for unknown codes.

D8 – Architecture Diagram



The system enables end-to-end Retrieval-Augmented Generation **(RAG)** from document ingestion to intelligent responses. User inputs are processed through a Streamlit interface, where data is **ingested, cleaned, chunked, embedded,** and **stored** in a vector database for efficient retrieval.

A RAG engine **combines relevant context** with a **local LLM** to generate accurate explanations, actions, and visual outputs, which are presented back to the user through Streamlit.

THANK YOU